Model theory in LEAN and the Flypitch Project

Based on Han & van Doorn, ITP 2019 & CPP 2020

Section 1

Introduction

The Flypitch Project

- Goal: formalize model theory in Lean 3 and use it to prove the independence of the Continuum Hypothesis (CH)
- Two papers:
  - Han & van Doorn, ITP 2019: ZFC $\nvdash$ CH
  - Han & van Doorn, CPP 2020: ZFC $\nvdash \neg$CH
- Much of the formalization of model theory in Lean has been adapted with varying changes from the Flypitch Project.
- In the same way much of the formalization of sets in the Flypitch project has been adapted from the work of Carneiro in mathlib.

Introduction

Goal of this talk

Formalization of model theory and Boolean Valued models in the Flypitch Project VS LEAN and an overview of its application on forcing and the proof of the independence of CH.

The Continuum Hypothesis (CH)

There is no set X with

$$\aleph_0 < |X| < 2^{\aleph_0}.$$

Index

- First order logic and ZFC
- Boolean-valued models
- Sketch of the mathematical proof of $ZFC \nvdash \mathrm{CH}$ using forcing.

Section 2

First-Order Logic and ZFC

Languages

A *first-order language* $\mathcal{L}$ consists of:

- A set of *function symbols* $\{f_i\}$, each with an arity $n_i \in \mathbb{N}$
  Example: $+$ (arity 2), 1 (arity 0)
- A set of *relation symbols* $\{R_j\}$, each with an arity $m_j \in \mathbb{N}$
  Example: $<$ (arity 2), $\in X$ (arity 1)

```
structure Language where
  Functions : ℕ → Type u
  Relations : ℕ → Type v
```

Terms

Terms built inductively from variables, constants, and function application. Formally: the smallest set T such that

- every variable x_i is a term
- if f has arity n and $t_1, \ldots, t_n \in T$, then $f(t_1, \ldots, t_n) \in T$

```
inductive preterm : ℕ → Type u
| var {} : ∀ (k : ℕ), preterm 0
| func : ∀ {l : ℕ} (f : L.functions l), preterm l
| app : ∀ {l : ℕ} (t : preterm (l + 1)) (s : preterm 0),
  preterm l
```

Flypitch implementation

```
preterm.app
  (preterm.app
    (preterm.func (L.functions 2).f := f(_,_))
    (preterm.var 1 := x_1))
  (preterm.var 0 := x_0)
```

Terms

```
inductive preterm : ℕ → Type u
| var : ∀ (k : ℕ), preterm 0
| func : ∀ {l : ℕ} (f : L.functions l), preterm l
| app : ∀ {l : ℕ} (t : preterm (l + 1)) (s : preterm 0),
  preterm l
```

Flypitch implementation

```
inductive Term (α : Type u') : Type max u u'
| var : α → Term α
| func : ∀ {l : ℕ} (_f : L.Functions l) (_ts : Fin l →
  Term α), Term α
```

mathlib implementation

Difference between the two implementations

- Difference in the indexing of variables:
  - Flypitch implementation uses a single type of terms, with a natural number indexing the number of free variables.
  - Mathlib implementation uses a family of types of terms, indexed by the type of free variables. This gives them more freedom to work with different types of free variables, but makes it more difficult to work with quantifiers as we will see later.
- Mathlib doesn't implement partial terms as they ended up not being needed for any proof in Flypitch (even if they were used from some in the original project).
- Because the Mathlib implementation applies all variables to a function at once they use "Fin l $\to$ Term α" as a function from slots of "f" to terms which removes the recursivity of the Flypitch implementation.

Formulas

Formulas are what we understand as a statement in first-order logic. we built them using terms, relations of the language and the standard logical operations

```
inductive preformula : ℕ → Type
| true : preformula 0
| false : preformula 0
| equal : (term L)  → (term L) → preformula 0
| rel : ∀ {n : nat}, L.relations n → preformula n
| apprel : ∀ {n : nat}, preformula (n + 1) → (term L) →
 preformula n
| imp : preformula 0 → preformula 0 → preformula 0
| all : preformula 0 → preformula 0
```

Flypitch implementation

We can write

$$\forall x_0\ \forall x_1 (x_0 \in \mathcal{P}(x_1) \iff (\forall x_2 (x_0 \in x_1 \implies x_0 \in x_2)))$$

```
∀' ∀' (&'0 ∈' P' &'1 <-> (∀' (&'0 ∈' &'1 ⟹ &'0 ∈' &'2)))
```

Formulas

```
inductive BoundedFormula  (α : Type u') : ℕ → Type max u
  v u'
| falsum {n} : BoundedFormula α n
| equal {n} (t₁ t₂ : L.Term (α ⊕ (Fin n))) :
  BoundedFormula α n
| rel {n l : ℕ} (R : L.Relations l)
  (ts : Fin l → L.Term (α ⊕ (Fin n))) : BoundedFormula α
   n
| imp {n} (f₁ f₂ : BoundedFormula α n) :
  BoundedFormula α n
| all {n} (f : BoundedFormula (n + 1)) :
  BoundedFormula α n
```

Mathlib implementation

In here α indexes the free variables while n indexes the bound variables.

Difference in the implementations

'all' takes the first unbound numbered variable and binds it. This is simple in the Flypitch implementation as all formulas are in the same type. In Mathlib, we have to change the indexing of the free variables to account for the new bound variable by working with $\alpha \oplus (\text{Fin } n)$.

Notice that both implementations only implement $\Rightarrow$, $\forall$ and $\bot$ as logical connectives. The other connectives are defined in terms of these three.

Extra elements

```
notation `⊥` := fol.bounded_preformula.bd_falsum
infix ` ≃ `:88 := fol.bounded_preformula.bd_equal
infixr ` ⟹ `:62 := fol.bounded_preformula.bd_imp
def bd_not {n} (f : bounded_formula L n) :
  bounded_formula L n := f ⟹ ⊥
prefix `~`:max := fol.bd_not
def bd_and {n} (f₁ f₂ : bounded_formula L n) :
  bounded_formula L n := ~ (f₁ ⟹ ~ f₂)
infixr ` ⊓ ` := fol.bd_and
def bd_or {n} (f₁ f₂ : bounded_formula L n) :
  bounded_formula L n := ~ f₁ ⟹ f₂
infixr ` ⊔ ` := fol.bd_or
def bd_biimp {n} (f₁ f₂ : bounded_formula L n) :
  bounded_formula L n := (f₁ ⟹ f₂) ⊓ (f₂ ⟹ f₁)
infix ` <=> `:61 := fol.bd_biimp
prefix `∀'`:110 := fol.bounded_preformula.bd_all
def bd_ex {n} (f : bounded_formula L (n+1)) :
  bounded_formula L n := ~ (∀' ( ~ f))
prefix `∃'`:110 := fol.bd_ex
```

Proofs

Proofs are built inductively from axioms and inference rules.

```
inductive prf : set (formula L) → formula L → Type u
| axm     {Γ A} (h : A ∈ Γ) : prf Γ A
| impI    {Γ} {A B} (h : prf (insert A Γ) B) :
    prf Γ (A =⇒ B)
| impE    {Γ} (A) {B} (h1 : prf Γ (A =⇒ B))
    (h2 : prf Γ A) : prf Γ B
| falsumE {Γ} {A} (h : prf (insert ~A Γ) ⊥) : prf Γ A
| allI    {Γ A} (h : prf ((λ f, f ↑ 1) 00 Γ) A) :
    prf Γ (∀0 A)
| allE2   {Γ} A t (h : prf Γ (∀0 A)) : prf Γ (A[t // 0])
| ref     (Γ t) : prf Γ (t ′ t)
| subst2  {Γ} (s t f) (h1 : prf Γ (s ′ t))
    (h2 : prf Γ (f[s // 0])) : prf Γ (f[t // 0])
```

Given a set of axioms Γ and a formula A, we define $\Gamma \vdash A$.

```
nonempty (prf Γ A)
```

Proofs

- Proofs are not implemented in Mathlib as there has been no interest in proof theory in Mathlib. Nevertheless, Flypitch implementation works for its uses and it may be able to be adapted into Mathlib.
- It is defined in Flypitch because it is required for the proof of the CH.
- Notice the difference between prf and the veracity of a prop in Lean is that prf is a proof tree and as such it has information about the proof and not only about the veracity of the statement.

Structures

Formulas are just a collection of symbols. It is require additional information to give them a truth value.

An $\mathcal{L}$-*structure* $\mathcal{M}$ gives meaning to the symbols of the language $\mathcal{L}$ by assigning a set M to the domain of discourse and interpreting each function and relation symbol as a function or relation on M.
Example: The structure $\mathcal{M} = (\mathbb{R}, +, 0)$ interprets the symbols of the language of groups $\mathcal{L} = (+, 0)$, $(\mathbb{R}^*, \cdot, 1)$ and $(\mathbb{Z}, +, 0)$ are also $\mathcal{L}$-structures.

```
class Structure  (L : Language) (M : Type w) where
  funMap : ∀ {n}, L.Functions n → (Fin n → M) → M := by
    exact fun {n} => isEmptyElim
  RelMap : ∀ {n}, L.Relations n → (Fin n → M) → Prop :=
    by exact fun {n} => isEmptyElim
```

We write $\mathcal{M} \models A$ if A is true in $\mathcal{M}$.

Structures

- Structures are not defined in Flypitch as they work with Boolean-valued structures which are a generalization of structures.
- Structure defines functions as Fin $n \to M \to M$ instead of $M^n \to M$ Which uses a similar trick as the one used to define term and formula in Mathlib.
- The definition of relations sends them to elements of time prop which contains the logical relations of the operators and contains the properties that make a structure.
- Structure is defined as a class because it is a property many mathematical objects define naturally and its checking process is simple due to Prop handling the heavy work.

completness theorem

Theorem

Let $\mathcal{L}$ be a first-order language, Γ a set of $\mathcal{L}$-sentences, and A an $\mathcal{L}$-sentence. Then, $\Gamma \vdash A$ if and only if for any $\mathcal{M}$ such that $\mathcal{M} \models \Gamma$ then $\mathcal{M} \models A$.

Language of ZFC

ZFC is a first-order theory in the language of set theory $\mathcal{L}_{\in}$ with a single binary relation symbol $\in$.

```
inductive ZFC_rel : ℕ → Type
| ε : ZFC_rel 2

def L_ZFC : Language :=
⟨λ_ => Empty, ZFC_rel⟩
```

Axioms of ZFC

ZFC is a first-order theory in the language of set theory $\mathcal{L}_\in$ with a single binary relation symbol $\in$.

```
inductive ZFC_rel : ℕ → Type 1
| ε : ZFC_rel 2

inductive ZFC_func : ℕ → Type 1
| emptyset : ZFC_func 0
| pr : ZFC_func 2
| ω : ZFC_func 0
| P : ZFC_func 1
| Union : ZFC_func 1

def L_ZFC : Language :=
{ functions := ZFC_func,
  relations := ZFC_rel }
```

Language conservatively extended with function symbols $\emptyset, (-,-), \omega, \mathcal{P}(-), \bigcup(-)$ for convenience.

ZFC

```
axiom_of_emptyset := ∀ x, x ∉ ∅
axiom_of_ordered_pairs := ∀ x y z w, (x, y) = (z, w) ↔ x = z ∧ y = w
axiom_of_extensionality := ∀ x y, (∀ z, (z ∈ x ↔ z ∈ y)) → x = y
axiom_of_union := ∀ u x, x ∈ ⋃ u ↔ ∃ y ∈ u, x ∈ y
axiom_of_powerset := ∀ z y, y ∈ P(z) ↔ ∀ x ∈ y, x ∈ z
axiom_of_infinity := ∅ ∈ ω ∧ (∀ x ∈ ω, ∃ y ∈ ω, x ∈ y) ∧ (∃ α, Ord(α) ∧ ω = α) ∧
  ∀ α, Ord(α) → (∅ ∈ α ∧ ∀ x ∈ α, ∃ y ∈ α, x ∈ y) → ω ⊆ α
axiom_of_regularity := ∀ x, x ≠ ∅ → ∃ y ∈ x, ∀ z ∈ x, z ∉ y
zorns_lemma := ∀ z, z ≠ ∅ → (∀ y, (y ⊆ z ∧ ∀ x₁ x₂ ∈ y, x₁ ⊆ x₂ ∨ x₂ ⊆ x₁) → (⋃y) ∈ z) →
  ∃ m ∈ x, ∀ x ∈ z, m ⊆ x → m = x
axiom_of_collection(φ) := ∀ p ∀ A, (∀ x ∈ A, ∃ y, φ(x,y,p)) →
  (∃ B, (∀ x ∈ A, ∃ y ∈ B, φ(x,y,p)) ∧ ∀ y ∈ B, ∃ x ∈ A, φ(x,y,p))

epsilon_transitive(z) := ∀ x, x ∈ z ⟹ x ⊆ z
epsilon_trichotomy(z) := ∀ x y ∈ z, x = y ∨ x ∈ y ∨ y ∈ x
epsilon_wellfounded(z) := ∀ x, x ⊆ z ⟹ x ≠ ∅ → ∃ y ∈ x, ∀ w ∈ x, w ∉ y
Ord(z) := epsilon_trichotomy(z) ∧ epsilon_wellfounded(z) ∧ epsilon_transitive(z)
```

```
def axiom_of_emptyset : Formula L_ZFC := ∀' (~(&0 ∈' ∅'))
```

pSet

- Lean is based on type theory, not set theory. Ideally we want to encode sets as types.
- Aczel–Werner encoding of sets
- Carneiro added the encoding to mathlib.

```
inductive PSet : Type (u + 1)
| mk (α : Type u) (A : α → PSet) : PSet
```

In here α indexes the element in the set and A is a function which returns the element of the set indexed by α. (for example $\{\emptyset\}$ could be represented by PSet.mk unit (fun _ => PSet.mk emptyset)).

Because the definition indexes by position it needs an equivalent relation which forgets the index of the elements

```
def Equiv : PSet → PSet → Prop
| ⟨_, A⟩, ⟨_, B⟩ => (∀ a, ∃ b, Equiv (A a) (B b)) ∧ (∀
  b, ∃ a, Equiv (A a) (B b))
```

Section 3

Boolean-Valued Models

Boolean algebra

Definition

A *Boolean algebra* is a set B with operations $\wedge$ (meet), $\vee$ (join), $\neg$ (complement), and constants $\top, \bot$ satisfying:

- **Commutativity**
  $a \wedge b = b \wedge a$
- **Associativity**
  $a \wedge (b \wedge c) = (a \wedge b) \wedge c$
- **Distributivity**
  $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$
- **Identity**
  $a \wedge \top = a, \quad a \vee \bot = a$
- **Complement**
  $a \wedge \neg a = \bot, \quad a \vee \neg a = \top$

The Partial Order

Define $a \leq b$ iff $a \wedge b = a$. Then $\bot \leq a \leq \top$ for all a.

Complete Boolean Algebras

Definition

A *complete Boolean algebra* $\mathscr{B}$ is a Boolean algebra in which every subset has an infimum $\bigsqcap$ and supremum $\bigsqcup$.

Example

$\mathbf{2} = \{\bot, \top\}$: classical truth values

Example of boolean algebra

Definition

U is *regular open* in space X if $U = (U^{\perp})^{\perp}$, where $U^{\perp}$ is the interior of the complement of $\overline{U}$. $\mathrm{RO}(X)$ (The regular open sets of X) is a complete Boolean algebra.

- **Commutativity**
  $a \cup b = b \cup a$
- **Associativity**
  $a \cup (b \cup c) = (a \cup b) \cup c$
- **Distributivity**
  $a \cup (b \cap c) = (a \cup b) \cap (a \cup c)$
- **Identity**
  $a \cap X = a, \quad a \cup \emptyset = a$
- **Complement**
  $a \cap a^{\perp} = \emptyset, \quad a \cup a^{\perp} = X$

Boolean-valued models

- A *Boolean-valued model* is a structure $\mathcal{M}$ in which formulas are interpreted as elements of a complete Boolean algebra $\mathscr{B}$.
- The truth value of a formula φ in $\mathcal{M}$ is denoted by $[\![\varphi]\!]_{\mathcal{M}} \in \mathscr{B}$.
- The interpretation of logical connectives and quantifiers is given by the operations of the Boolean algebra:

$$
\begin{aligned}
[\![\varphi \wedge \psi]\!]_{\mathcal{M}} &= [\![\varphi]\!]_{\mathcal{M}} \sqcap [\![\psi]\!]_{\mathcal{M}} \\
[\![\varphi \vee \psi]\!]_{\mathcal{M}} &= [\![\varphi]\!]_{\mathcal{M}} \sqcup [\![\psi]\!]_{\mathcal{M}} \\
[\![\neg\varphi]\!]_{\mathcal{M}} &= \neg [\![\varphi]\!]_{\mathcal{M}} \\
[\![\forall x\, \varphi(x)]\!]_{\mathcal{M}} &= \bigsqcap_{m} [\![\varphi(m)]\!]_{\mathcal{M}} \\
[\![\exists x\, \varphi(x)]\!]_{\mathcal{M}} &= \bigsqcup_{m} [\![\varphi(m)]\!]_{\mathcal{M}}
\end{aligned}
$$

Boolean-valued models

```
structure bStructure :=
(carrier : Type u)
(fun_map : ∀{n}, L.functions n → dvector carrier n →
 carrier)
(rel_map : ∀{n}, L.relations n → dvector carrier n → β)
(eq : carrier → carrier → β)
(eq_refl : ∀ x, eq x x = ⊤)
(eq_symm : ∀ x y, eq x y = eq y x)
(eq_trans : ∀{x} y {z}, eq x y ⊓ eq y z ≤ eq x z)
(fun_congr : ∀{n} (f : L.functions n) (x y : dvector
 carrier n),
  (x.map2 eq y).fInf ≤ eq (fun_map f x) (fun_map f y))
(rel_congr : ∀{n} (R : L.relations n) (x y : dvector
 carrier n),
  (x.map2 eq y).fInf ⊓ rel_map R x ≤ rel_map R y)
```

The definition of a Boolean-valued structure is similar to that of a classical structure, but with the addition of a complete Boolean algebra $\mathscr{B}$ and the interpretation of formulas as elements of $\mathscr{B}$. This adds all the checkings Prop deals with in the classical definition and makes it inefficient to be defined as a class. It is only used twice in the Flypitch project which is the other reason it is defined as an structure.

Boolean-Valued Soundness Theorem

Theorem

If $\Gamma \vdash \varphi$, then for every $\mathscr{B}$ and every $\mathscr{B}$-valued structure $\mathcal{M}$:

$$\prod_{\gamma \in \Gamma} [\![\gamma]\!]_{\mathcal{M}} \leq [\![\varphi]\!]_{\mathcal{M}}.$$

Why is this useful?

- $\mathcal{M} \models_{\mathscr{B}} \gamma$ for $\gamma \in \mathsf{ZFC}$, $[\![\mathsf{CH}]\!]_{\mathcal{M}} < \top \implies \mathsf{ZFC} \nvdash \mathsf{CH}$
- $\mathcal{M} \models_{\mathscr{B}} \gamma$ for $\gamma \in \mathsf{ZFC}$, $[\![\neg\mathsf{CH}]\!]_{\mathcal{M}} < \top \implies \mathsf{ZFC} \nvdash \neg\mathsf{CH}$

Proved by structural induction on proof trees.

From pSet to bSet

Aczel–Werner encoding of V:

```
inductive pSet : Type (u+1)
| mk (a : Type u) (A : a -> pSet) : pSet
```

Generalize with a third field assigning Boolean truth-values:

```
inductive bSet (B : Type u) [cBA B] : Type (u+1)
| mk (a : Type u) (A : a -> bSet) (b : a -> B) : bSet
```

Because all the information is contained in b, the equivalent relation is not necessary in this structure which makes it easier to work with in Lean.

Boolean-Valued Equality and Membership

Defined with implementation:

```
def bv_eq'' : ∀ (x y : bSet 𝔹), 𝔹
| ⟨α, A, B⟩ ⟨α', A', B'⟩ :=
    (⊓a : α, B a ⟹ ⊔a', B' a' ⊓ bv_eq (A a) (A' a')) ⊓
      (⊓a' : α', B' a' ⟹ ⊔a, B a ⊓ bv_eq (A a) (A' a'))


def mem : bSet 𝔹 → bSet 𝔹 → 𝔹
| a (mk α' A' B') := ⊔a', B' a' ⊓ a =ᴮ A' a'
```

We denote this structure by $\mathbf{V}^{\mathscr{B}}$.

Theorem (Fundamental Theorem of Forcing)

For every complete Boolean algebra $\mathscr{B}$, $\mathbf{V}^{\mathscr{B}} \models_{\mathscr{B}}$ ZFC*: every axiom* σ *of* ZFC *has* $[\![\sigma]\!]^{\mathscr{B}}_{\mathbf{V}} = \top$.

Why Boolean-valued models?

- Boolean-valued models are a generalization of classical structures.
- They are particularly useful in forcing arguments, where we want to construct models of set theory that satisfy certain properties.
- Both arguments can be built with the same framework: $\mathscr{B}$-valued logic, $\mathscr{B}$-valued set theory, and the fundamental theorem of forcing.
- `bSet` is an inductive type (in a very Lean way) and avoids transfinite recursion and the quotient construction of `pSet`.

Section 4

The proof

Sketch of the proof

Theorem (Failure of CH in $\mathbf{V}^{\mathscr{B}}$)

Let $\mathscr{B} = \mathrm{RO}(2^{\aleph_2 \times \omega})$ and let $\mathbf{V}^{\mathscr{B}}$ be the corresponding Boolean-valued universe. Then

$$[\![\neg \mathsf{CH}]\!]_{\mathscr{B}} = \top.$$

That is, $\mathbf{V}^{\mathscr{B}}$ satisfies $\neg$CH.

Solution.

$\neg$CH asserts the existence of a set strictly between ω and $\mathcal{P}(\omega)$ in cardinality. We show $\mathbf{V}^{\mathscr{B}}$ witnesses this with $\check{\aleph}_1$.

Claim 1: $[\![\check{\omega} \text{ is strictly smaller than } \check{\aleph}_1]\!] = \top$.

Claim 2: $[\![\check{\aleph}_1 \text{ is strictly smaller than } \check{\aleph}_2]\!] = \top$.

Claim 3: $[\![\check{\aleph}_2 \hookrightarrow \mathcal{P}(\check{\omega})]\!] = \top$.

$$[\![\check{\omega} \prec \check{\aleph}_1 \prec \check{\aleph}_2 \leq \mathcal{P}(\check{\omega})]\!]_{\mathscr{B}} = \top,$$

Main theorem

Theorem (Unprovability of CH)

If ZFC *is consistent, then* $\mathrm{ZFC} \nvdash \mathrm{CH}$. *Equivalently,* $\mathrm{ZFC} + \neg\mathrm{CH}$ *is consistent relative to* ZFC.

Solution.

Assume ZFC is consistent. We have established:

1. We know $\mathbf{V}^{\mathscr{B}}$ is a $\mathscr{B}$-valued model of ZFC: $[\![\sigma]\!]_{\mathscr{B}} = \top$ for every axiom σ of ZFC.
2. and by the previous theorem, $[\![\neg\mathrm{CH}]\!]_{\mathscr{B}} = \top$.

By the Boolean-valued soundness theorem, if $\mathrm{ZFC} \vdash \mathrm{CH}$, then

$$\top = \bigsqcap_{\sigma \in \mathrm{ZFC}} [\![\sigma]\!]_{\mathscr{B}} \leq [\![\mathrm{CH}]\!]_{\mathscr{B}}.$$

But $[\![\mathrm{CH}]\!]_{\mathscr{B}} \leq \neg[\![\neg\mathrm{CH}]\!]_{\mathscr{B}} = \neg\top = \bot$, a contradiction. Hence $\mathrm{ZFC} \nvdash \mathrm{CH}$.

Section 5

Ending

Thank you

Any questions?

Bibliography

[1] Peter Aczel et al. "The type theoretic interpretation of constructive set theory". In: *Journal of Symbolic Logic* 49.1 (1984).

[2] Jesse Michael Han and Floris van Doorn. "A formal proof of the independence of the continuum hypothesis". In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. 2020, pp. 353–366.

[3] Jesse Michael Han and Floris van Doorn. "A formalization of forcing and the unprovability of the continuum hypothesis". In: *arXiv preprint arXiv:1904.10570* (2019).

[4] Richard Mansfield and John Dawson. "Boolean-valued set theory and forcing". In: *Synthese* 33.1 (1976), pp. 223–252.

[5] *mathlib4 documentation*. Lean community mathlib4 reference documentation. URL: https://leanprover-community.github.io/mathlib4_docs (visited on 06/07/2026).